

SONAR · ВЕБ-ПЛАТФОРМА ПОТОКОВОГО ВИДЕО

Сценарий выступления

Защита итоговой работы · 29 мая 2026 года

Лукинов Виталий Леонидович, доцент

Андреев Андрей Валерьевич, старший преподаватель

Текст читки на 10 минут, ~1500 слов (~150 слов в минуту).

Маркеры слайдов в скобках, для удобства синхронизации с перелистыванием.

(СЛАЙД 01 · ≈20 СЕК)

Титул

Уважаемые члены комиссии, разрешите представить итоговую работу по программе. Тема: веб-платформа потокового видео с синхронным совместным просмотром, рабочее название Sonar. Авторы: доцент Лукинов Виталий Леонидович и старший преподаватель Андреев Андрей Валерьевич.

(СЛАЙД 02 · ≈50 СЕК)

Цель и задачи

Цель работы: спроектировать и реализовать веб-платформу потокового просмотра видео. Пользователи могут загружать свои ролики или брать их с внешних источников, искать в каталоге, смотреть в браузерном плеере; дополнительно поддерживается совместный просмотр в общих комнатах с чатом и вопросами от зрителей.

Для достижения цели поставлены пять задач. Первая: проанализировать существующие решения и сформировать требования. Вторая: спроектировать архитектуру системы с разделением на бэкенд, фронтенд и хранилище. Третья: реализовать каталог видео, плеер и личный профиль пользователя. Четвёртая: добавить возможность совместного просмотра в комнатах с чатом и вопросами. Пятая: покрыть систему тестами и развернуть боевой стенд.

(СЛАЙД 03 · ≈25 СЕК)

Распределение ролей

Работа выполнена в паре. За Лукиновым Виталием Леонидовичем закреплены две задачи: моделирование архитектуры информационной системы и разработка серверной бизнес-логики. За Андреевым Андреем Валерьевичем: прототипирование и разработка пользовательского интерфейса, а также тестирование и интеграционные работы.

(СЛАЙД 04 · ≈30 СЕК)

Средства реализации

Технологический стек подобран под потоковое видео в браузере и общение в комнатах в реальном времени. Серверная часть: Django пятой версии и Django Channels четвёртой версии, каналы используются для WebSocket-соединений в комнатах совместного просмотра. Хранилище: PostgreSQL для постоянных данных и Redis в роли брокера сообщений между процессами Django. Загруженные пользователями ролики отдаются как MP4-файлы, внешние источники парсятся через yt-dlp и могут быть как MP4, так и HLS. Клиент написан на React девятнадцатой версии, плеер построен на библиотеке hls.js, которая корректно работает с обоими форматами. Сборка и развёртывание: Docker и обратный прокси Caddy.

(СЛАЙД 05 · ≈30 СЕК)

Планирование работ

Календарный план уложен в один месяц. В первые дни мая собирались требования. Затем три недели велись параллельные ветки: серверная, где реализовывались REST API и стриминг через WebSocket, и клиентская, где собирались каркас интерфейса, плеер, чат и панель вопросов. Заключительная неделя отведена на тестирование и развёртывание боевого стенда.

(СЛАЙД 06 · ≈50 СЕК)

Архитектура системы

Системная диаграмма приведена в нотации C4 на уровне контейнеров. Пользователь работает через браузер и общается с двумя контейнерами серверной части. REST API по протоколу HTTPS обслуживает основной функционал платформы: каталог видео, загрузку, профиль и авторизацию. WebSocket-каналы поверх HTTPS обслуживают дополнительную возможность совместного просмотра: события синхронизации плеера, сообщения чата и вопросы зрителей. WebSocket-события проходят через брокер Redis в режиме публикация-подписка, чтобы несколько процессов Django могли мгновенно обмениваться состоянием комнаты. Метаданные пользователей, видео и комнат хранятся в PostgreSQL и читаются через ORM. Сами видеофайлы лежат на медиа-томе и отдаются обратным прокси Caddy как статика.

(СЛАЙД 07 · ≈40 СЕК)

Диаграммы кода

На уровне доменной модели выделено пять ключевых классов. Пользователь владеет профилем, загруженными видео и созданными комнатами. Видео является центральной сущностью каталога: хранит метаданные и ссылку на медиа-файл. Комната объединяет нескольких участников и связана с тем видео, которое в ней проигрывается совместно. К комнате привязаны сообщения чата и вопросы аудитории. Базовый сценарий: пользователь загружает или находит видео в каталоге и смотрит его в своём плеере. Расширенный сценарий: создаёт комнату и приглашает других смотреть вместе.

(СЛАЙД 08 · ≈30 СЕК)

Диаграмма компонентов UI

Клиентская часть разбита на дерево React-компонентов. Корневые страницы: главная с лентой видео, страница комнаты просмотра и личный профиль. Каждая страница опирается на переиспользуемые компоненты: карточка видео, плеер на основе hls.js, панель чата и панель вопросов с возможностью голосования за чужой вопрос.

(СЛАЙД 09 • ≈30 СЕК)

Главная: макет и реализация

Слева приведён Figma-макет ленты, справа реальный интерфейс боевого стенда. В шапке логотип Sonar и пункты навигации: каталог, переход в комнату по ссылке, загрузка видео, аватар и имя пользователя. Основную часть экрана занимает сетка карточек по четыре в ряд: на каждой превью с длительностью ролика, название, автор и число просмотров. Макет повторяет ту же структуру в упрощённом схематичном виде.

(СЛАЙД 10 • ≈35 СЕК)

Совместный просмотр: макет и реализация

В большей левой части слайда плеер с соотношением шестнадцать к девяти, под ним название ролика, описание и кнопка обновления плеера. Справа панель с тремя вкладками: чат, вопрос-ответ и список зрителей. Сообщения чата можно лайкать. На вкладке вопрос-ответ зрители оставляют вопросы и голосуют за чужие, вопросы автоматически сортируются по количеству голосов, чтобы ведущий сначала видел самые интересные. Внизу комнаты кнопка копирования ссылки-приглашения, через которую можно позвать друзей в общий просмотр. Макет слева показывает ту же раскладку в схематичном виде.

(СЛАЙД 11 · ≈25 СЕК)

Профиль: макет и реализация

Сверху карточка пользователя: аватар, имя и подпись-тег, поле имени для отображения в чате и кнопка выхода со всех устройств. Ниже раздел «Мои видео» с сеткой собственных роликов: на каждой карточке превью, название, автор и кнопки изменения и удаления для управления своим контентом. Макет слева показывает те же блоки упрощённо.

(СЛАЙД 12 · ≈55 СЕК)

Совместный просмотр: синхронизация плеера

Когда зрители смотрят вместе в одной комнате, плееры у всех должны показывать одну и ту же секунду, иначе обсуждение в чате теряет смысл. Алгоритм синхронизации следующий. Когда участник нажимает «воспроизведение», его клиент посылает серверу событие через WebSocket. Сервер фиксирует серверное время этого события и рассылает его всем участникам комнаты через Redis-канал. Получив сообщение, остальные клиенты вычисляют ожидаемую позицию плеера как серверную позицию плюс разницу между серверным и текущим клиентским временем. Если расхождение меньше трёхсот миллисекунд, клиент незаметно подстраивает скорость воспроизведения в пределах плюс-минус пяти процентов, чтобы постепенно сравняться. Если расхождение больше секунды, плеер делает резкий переход к нужной позиции.

(СЛАЙД 13 • ≈40 СЕК)

Пример реализации синхронизации

На слайде приведён фрагмент серверного обработчика события «воспроизведение» в комнате совместного просмотра. Channels-консьюмер принимает входящее сообщение, валидирует право участника управлять плеером, проставляет авторитетную метку серверного времени и рассылает событие в группу комнаты. На клиенте при получении этого пакета вызывается функция корректировки, которая реализует ту самую мягкую подстройку скорости, о которой шла речь на предыдущем слайде.

(СЛАЙД 14 • ≈35 СЕК)

Тестирование

Тестовое покрытие построено на двух уровнях. На бэкенде используется pytest с подключением к настоящей базе и настоящему Redis. Проверяются сценарии работы с каталогом и загрузкой видео, авторизация, а также сценарии комнат совместного просмотра: создание, вход второго пользователя, рассылка событий play, pause и seek, граничные случаи участник вышел и вернулся, видео завершилось. На фронтенде используется Vitest для модулей синхронизации, компонентных тестов плеера, кнопок и форм. Покрыты ключевые сценарии работы с каталогом, плеером и комнатой.

(СЛАЙД 15 • ≈35 СЕК)

Интеграция и развёртывание

Развёртывание полностью автоматизировано. Все сервисы упакованы в Docker: контейнер с Django и Channels, контейнер с PostgreSQL, контейнер с Redis и контейнер обратного прокси Caddy. Caddy самостоятельно получает и обновляет TLS-сертификат от Let's Encrypt и проксирует трафик HTTPS и WebSocket на бэкенд. Развёртывание стенда сводится к запуску одного сценария `deploy.sh`, который тянет свежий код, поднимает контейнеры и применяет миграции базы.

(СЛАЙД 16 • ≈40 СЕК)

Заключение

Цель работы достигнута. Платформа Sonar спроектирована, реализована и развёрнута на боевом стенде. По задачам: требования собраны, архитектура описана на уровне контейнеров и доменных классов, серверная часть построена поверх Django и Channels, клиентский интерфейс собран на React с плеером на `hls.js`, реализован сценарий совместного просмотра в комнатах с синхронизацией, чатом и вопросами, и система покрыта автоматическими тестами. Платформа решает прикладную задачу потокового просмотра видео в браузере и может развиваться в сторону живых трансляций.

(СЛАЙД 17 • ≈15 СЕК)

Вопросы и ответы

Спасибо за внимание. Готовы ответить на ваши вопросы.